```
    initContract: function() {
      ...
        App.contracts.Payment = new App.web3.eth.Contract(data.abi,
                    data.networks[App.network_id].address, {});        ❸
...      })

  ...  },

    populateAddress : function(){
      ..
        new Web3(App.url).eth.getAccounts((err, accounts) => {         ❹
...    },

    handleSignedMessage:function(receiver,amount){
   ...
      var weiamount=App.web3.utils.toWei(amount, 'ether');
      ...
    },

    constructPaymentMessage:function(contractAddress, weiamount) {
      return App.web3.utils.soliditySha3(contractAddress,weiamount)    ❺
    },

    signMessage:function (message,amount) {
      web3.personal.sign(message, web3.eth.defaultAccount, function(err,
                                      signedMessage)                   ❻
{
...    },

handleTransfer:function(amount,signedMessage){

        if(App.web3.utils.isHexStrict(signedMessage)){
        var weiamount=App.web3.utils.toWei(amount,'ether')             ❼
        var amount=App.web3.utils.toHex(weiamount)                     ❼
        ...}
```

❶ Data for initializing application App

❷ Web3 initialized with Ganache, the web3 provider here

❸ Connect to the smart contract ABI and address.

❹ Get accounts created in the web3 provider Ganache.

❺ web3.utils used to hash the message

❻ web3.personal to sign it

❼ web3.utils used to convert the payment to wei

app.js in MPC-Dapp demonstrates the use of a variety of the web3 packages, from web3.eth to web3.utils. Observe the structural flow of the various functions in listing 7.2. Next, let’s